

ДЗ: реализация 6-битного MMU на Rust

Нужно написать эмулятор аппаратного блока управления памятью (MMU) для игрушечной 6-битной архитектуры. MMU переводит виртуальный адрес (VA, virtual address) в физический адрес (PA, physical address).

Рыба кода	https://gist.github.com/kulinsky/39cee1674cbdf54432734bfe78e9ce0d
Проверка	cargo test

Технические условия системы

Размер адреса

Адрес занимает 6 бит. Это значит, что вся адресуемая память занимает: $2^6 = 64$ байта

Допустимые виртуальные адреса: $0..=63$

Если в функцию `translate` передан адрес больше 63, нужно вернуть ошибку

Разбиение виртуального адреса

Размер страницы - 8 байт. младшие 3 бита адреса используются как смещение внутри страницы. Старшие 3 бита используются как индекс виртуальной страницы.

6-битный виртуальный адрес:

VPN	OFFSET
3 бита	3 бита

VPN - virtual page number, номер виртуальной страницы

OFFSET - смещение внутри страницы

Пример: VA = 0b101_011

VPN = 0b101 = 5

OFFSET = 0b011 = 3

Формат Page Table Entry

Каждый элемент `page_table` - это один байт со следующим форматом:

```
бит:  7 6 5 4 3 2 1 0
      - - P P P U W P
          | | | | | |
          | | | | | +-- Present
          | | | | +---- Writable
          | | | +----- User
          +---+----- Physical Page Number
```

Биты	Назначение
5..3	Номер физической страницы
2	Флаг U, User
1	Флаг W, Writable
0	Флаг P, Present
6..7	Не используются в этом задании

Что должна делать функция `translate`

```
pub fn translate(&self, va: u8, is_write: bool) -> Result
```

должна выполнить перевод виртуального адреса в физический.

`va` - 6-битный виртуальный адрес.

`is_write` - флаг операции:

Алгоритм работы

1. Проверить адрес
2. Получить номер виртуальной страницы
3. Получить смещение внутри страницы
4. Найти запись в таблице страниц
5. Проверить флаг `Present`
6. Проверить права доступа
7. Получить номер физической страницы
8. Собрать физический адрес